

Session 2 .

- 1.Install GitHub Copilot in VS Code and use it to generate a React functional component called PlaylistCard that displays a music playlist's name, creator, and song count.
- 2.Copy a React component that displays a Flipkart-style product card (with image, title, and price) into ChatGPT and ask it to explain any errors or warnings you see in your VS Code terminal.

Hint: Paste both your code and the exact error message into ChatGPT for best results.
- 3.Write a prompt for ChatGPT that asks it to refactor your React component for a Zomato restaurant listing to use a custom reusable hook for fetching restaurant data.
- 4.Use GitHub Copilot's auto-suggestions to refactor a React component that shows a list of YouTube video titles, moving the data fetching logic into a separate useVideos hook.
- 5.After using ChatGPT or Copilot to generate or refactor a React component, review the suggested code and list 2 improvements or corrections you would make before using it in a real project.

Hint: Think about variable names, missing prop types, or unnecessary code.

Answer:-

PlatlistCard.jsx

```
import React from "react";

const PlaylistCard = ({ name, creator, songCount }) => {
  return (
    <div className="playlist-card">
      <h2>{name}</h2>
      <p>Created by: {creator}</p>
      <p>Songs: {songCount}</p>
    </div>
  );
};

export default PlaylistCard;
```

2. Using ChatGPT to Explain React Errors

Flipkart-Style Product Card Component

```
import React from "react";

const ProductCard = ({ image, title, price }) => {
  return (
    <div className="product-card">
      <img src={image} alt={title} />
      <h3>{title}</h3>
      <p>₹{price}</p>
    </div>
  );
};
```

```

</div>
);
};

export default ProductCard;

```

3. Prompt to Refactor a Zomato Restaurant Listing Using a Custom Hook

Refactor my React component for a Zomato-style restaurant listing.

Requirements:

1. Move all restaurant data fetching logic into a custom hook called useRestaurants.
2. Keep the UI rendering logic inside the RestaurantList component.
3. Use React hooks and modern JavaScript.
4. Add loading and error states.
5. Return clean, reusable, and production-ready code.
6. Explain the changes made.

Here is my current component:

4. Refactoring a YouTube Video List Using a Custom Hook

```

import React, { useEffect, useState } from "react";

function VideoList() {
  const [videos, setVideos] = useState([]);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/posts")
      .then((res) => res.json())
      .then((data) => setVideos(data.slice(0, 10)));
  }, []);

  return (
    <div>
      <h2>YouTube Videos</h2>

      {videos.map((video) => (
        <p key={video.id}>{video.title}</p>
      ))}
    </div>
  );
}

export default VideoList;

```

useVideos.js

```
import { useEffect, useState } from "react";

const useVideos = () => {
  const [videos, setVideos] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/posts")
      .then((res) => res.json())
      .then((data) => {
        setVideos(data.slice(0, 10));
        setLoading(false);
      });
  }, []);

  return { videos, loading };
};

export default useVideos;
```

Refactored Documents

```
import React from "react";
import useVideos from "../useVideos";

function VideoList() {
  const { videos, loading } = useVideos();

  if (loading) {
    return <p>Loading videos...</p>;
  }

  return (
    <div>
      <h2>YouTube Videos</h2>

      {videos.map((video) => (
        <p key={video.id}>{video.title}</p>
      ))}
    </div>
  );
}

export default VideoList;
```

5. Improvements Before Using AI-Generated Code in Production

Add validation Props

```
import PropTypes from "prop-types";

PlaylistCard.propTypes = {
  name: PropTypes.string.isRequired,
  creator: PropTypes.string.isRequired,
  songCount: PropTypes.number.isRequired,
};
```

```
const [error, setError] = useState(null);

try {
  const response = await fetch(url);
  const data = await response.json();
} catch (err) {
  setError("Failed to fetch data");
}
```